

Superscalar Architecture

Week 4: Branch Prediction, Reservation Stations, and Milestone 2

May 31, 2026

Contents

Week 4 in 60 Seconds	4
1 Week 4 — Branch Prediction, Reservation Stations, and Milestone 2	6
2 Comprehensive Theory 1: Why Branches Are Catastrophic at Scale [~20 min]	8
2.1 The Branch Problem in One Sentence	8
2.2 The Cost of Waiting	8
2.3 The Fundamental Strategy: Speculation	8
2.4 The Economics of Speculation	9
3 Comprehensive Theory 2: Static Prediction and 1-Bit Dynamic [~20 min]	10
3.1 Static Prediction	10
3.2 Why Static Isn't Enough	10
3.3 The 1-Bit Predictor	10
3.4 The 1-Bit Predictor's Failure Mode	11
4 Comprehensive Theory 3: 2-Bit Counters, BHT, BTB, Speculation [~35 min]	12
4.1 The 2-Bit Saturating Counter	12
4.2 Tracing the 2-Bit Predictor on the Earlier Example	12
4.3 The Branch History Table (BHT)	13
4.4 The Branch Target Buffer (BTB)	13
4.5 Putting Them Together: The Front-End Speculation Loop	13
4.6 Speculation and the ROB	14
4.7 What PRAVAH Will Implement	14
5 Comprehensive Theory 4: The Reservation Station Micro-Architecture [~30 min]	16
5.1 The Mental Model: A Waiting Room	16
5.2 The Reservation Station Array	17
5.3 The Three Phases of an RS Cycle	17
5.4 A Worked Cycle-by-Cycle Trace	17
5.5 What Happens When Multiple RSs Are Wakeup-Ready?	19
5.6 What the RS Does NOT Do	19
6 Module: reservation_station.v [~1 hour]	20
6.1 Specification	20
6.2 Port List	20
6.3 Internal Structure	21
6.4 Implementation	22

6.5	Common Bugs to Watch For	25
7	Testbench: <code>tb.reservation_station.v</code> [~45 min]	26
7.1	What the Testbench Must Cover	26
7.2	Testbench Skeleton	26
7.3	Running the Testbench	31
8	The Design Decisions Document [~1 hour]	33
8.1	Required Content	33
8.1.1	Pipeline width	33
8.1.2	Register file sizing	33
8.1.3	Reservation stations	33
8.1.4	Reorder buffer	33
8.1.5	Functional units	33
8.1.6	Branch predictor	34
8.1.7	ISA subset	34
8.1.8	Memory model	34
8.1.9	What we are NOT building	34
8.2	Format of the Document	35
8.3	Why This Matters	35
8.4	Example Stub	35
9	Week 4 Deliverables — Milestone 2	37
A	Appendix: Solutions to Concept Checks	39

Week 4 in 60 Seconds

A one-page summary. This is **Milestone 2 week**. Theory load is medium; coding load is heavier than Week 3. Two parallel tracks: branch prediction (theory) and reservation stations (Verilog + theory).

Why branch prediction matters: A modern OoO pipeline has 30–100+ instructions in flight. Every branch is a fork in the road — the pipeline must *guess* which way to go and start fetching speculatively, because waiting for the branch to resolve would idle the entire machine. A 90% accurate predictor on a typical workload is the difference between a fast processor and a useless one.

The prediction hierarchy:

1. **Static prediction:** “always taken” or “always not taken.” Cheap, useful for loops (which are usually taken). Accuracy: ~60%.
2. **1-bit dynamic predictor:** Remember the last outcome per branch. Accuracy: ~80%.
3. **2-bit saturating counter:** The standard. Each branch has a 2-bit state machine. Mispredict once, you don’t immediately flip — you need to mispredict twice. Accuracy: ~90%.
4. **Branch History Table (BHT):** An array of 2-bit counters, indexed by PC. Many branches share the same table; collisions cause inaccuracy.
5. **Branch Target Buffer (BTB):** Stores the *target address* of taken branches so you can fetch from the predicted destination immediately, not after computing the target.

The reservation station: A “waiting room” for one instruction. It holds the instruction’s operation, plus the indices of its two source physical registers, plus a busy bit. Every cycle, the RS reads the PRF’s ready bits for its two sources. When both are ready, the RS’s instruction is **wakeup-ready**: it raises a hand to the issue logic. A priority encoder picks one wakeup-ready RS per cycle per functional unit and **issues** it: reads operand values from the PRF, sends them to the FU.

Key reframe from classical Tomasulo: In Week 2 we drew reservation stations with V_j/V_k (value) and Q_j/Q_k (tag) fields. With the modern PRF design, the RS only needs the source physical register indices — the values live in the PRF, and the “tag” is just the index. No V field needed in the RS.

Speculation in the ROB: When dispatch encounters a branch, the predictor guesses the direction; the front-end fetches from the predicted path; all speculative instructions are tagged with the branch’s ROB entry. When the branch resolves in EX, if the prediction was correct, do nothing. If wrong, flush all younger ROB entries (which we discussed in Week 3).

This week’s deliverables (Milestone 2):

- `reservation_station.v` — an array of 4 ALU reservation stations with PRF ready-bit snooping and wakeup-select logic.
- `tb_reservation_station.v` — self-checking testbench.
- `design_decisions.md` — a 1-page document locking down: issue width (2), RS count per FU, ROB size, ISA subset (RV32I subset of 12–15 instructions), FU mix (2 ALUs, 1 MUL, 1 LSU), branch predictor (2-bit BHT).
- ModelSim waveform showing two independent instructions issuing in successive cycles.

Suggested reading order: Sections 1–3 are branch-prediction theory. Section 4 is the RS micro-architecture. Section 5 is the Verilog. Section 6 is the testbench. Section 7 is the design-decisions document. Section 8 is the deliverables.

1 Week 4 — Branch Prediction, Reservation Stations, and Milestone 2

Goal

Two tracks running in parallel this week:

1. **Branch prediction theory:** understand why a modern OoO pipeline cannot function without it, and learn the standard mechanisms (2-bit counters, BHT, BTB, speculation/recovery). You will not implement a sophisticated branch predictor this week — that comes in Week 6 as part of integration. But the design-decisions document you write this week must specify which predictor you'll build.
2. **Reservation station Verilog:** implement the heart of the dynamic scheduler. The RS array is what makes out-of-order execution possible in hardware. Mentees who got Week 3 working will find this similar in spirit, but with more interesting control logic.

Mentee Work (9–10 hours)

- Read Sections 1–3 (branch prediction theory, ~2 hours). Concept Checks as you go.
- Read Section 4 (RS micro-architecture, ~45 min). Hand-trace the worked example.
- Read Sections 5–6 (Verilog implementation + testbench, ~1 hour).
- Code, simulate, verify the reservation station module (~4–5 hours).
- Write the design-decisions document (~1 hour).

Deliverables (Milestone 2)

- `rtl/reservation_station.v` — 4-entry ALU RS array with PRF snooping and wakeup-select.
- `tb/tb_reservation_station.v` — self-checking testbench covering at least 8 scenarios.
- `docs/design_decisions.md` — one page locking down the PRAVAH architectural parameters.
- `sim/waveforms/` — ModelSim screenshots of key test scenarios.
- Updated `learning_log.md`.

What We Will Look For (Milestone 2 Gate)

- RS correctly snoops the PRF ready bits each cycle without polling logic explosion (use combinational ready inputs from the PRF, not a tag-broadcast bus — consistent with the Week 3 design).
- When both sources of an RS go ready, the wakeup signal asserts within one cycle.
- Wakeup-select logic picks exactly one ready RS per cycle when multiple are ready (deterministic priority).
- Design-decisions document is specific enough to drive the next 4 weeks: exact numbers for RS count, ROB size, PRF size, ISA subset, predictor type.
- Plain Verilog only (no SystemVerilog).

2 Comprehensive Theory 1: Why Branches Are Catastrophic at Scale [~20 min]

2.1 The Branch Problem in One Sentence

*In an OoO pipeline that fetches 2 instructions per cycle and has 16+ instructions in flight, every branch creates a fork in the road, and the only way to keep the pipeline busy is to **guess** which side of the fork to take — before knowing the answer.*

Let's make this concrete with numbers.

2.2 The Cost of Waiting

Suppose we have a 6-stage pipeline (fetch, decode, rename, dispatch, execute, commit), 2-wide. We hit a branch. The branch resolves in the EX stage, which is cycle 5 after the branch was fetched.

If we wait for the resolution before fetching anything else, the front-end stalls for 5 cycles. At 2 instructions per cycle, that's $5 \times 2 = 10$ wasted instruction slots per branch.

In typical code, roughly 1 in every 5–6 instructions is a branch. So branches account for around 17% of all instructions. If every branch costs 10 wasted slots:

$$\text{wasted slots} = 0.17 \times 10 = 1.7 \text{ wasted slots per instruction}$$

Our 2-wide pipeline, instead of delivering $\text{IPC} = 2.0$, would deliver $\text{IPC} \approx 2.0 / (1 + 1.7) = 0.74$. **We made it slower than a single-issue scalar pipeline.**

This is not a small effect. Branches are the dominant performance bottleneck of OoO execution. Everything in this section is about not paying that cost.

2.3 The Fundamental Strategy: Speculation

The fix is simple to state: **don't wait. Guess.**

1. When the front-end sees a branch, it *predicts* the direction (taken or not taken) and the target address.
2. It immediately fetches and dispatches instructions from the predicted path.
3. These instructions enter the pipeline marked as **speculative** — they hold ROB entries, allocate physical registers, and even execute, but they have not committed.
4. When the branch resolves in EX, we compare the actual outcome to the prediction.
5. If correct: do nothing. Speculation is silent — the pipeline runs at full speed.
6. If wrong: **flush** all speculative work younger than the branch (this is what the ROB makes possible) and restart fetch from the correct path.

2.4 The Economics of Speculation

Speculation is a bet. We win if the prediction is correct (full-speed execution), we lose if it's wrong (10+ wasted cycles plus all the flush work).

Suppose the predictor is right 80% of the time and the misprediction penalty is 10 cycles. Then average penalty per branch:

$$0.20 \times 10 = 2 \text{ cycles per branch}$$

With branches at 17% of instructions, that's $0.17 \times 2 = 0.34$ cycles of penalty per instruction. IPC drops from 2.0 to $2.0 / (1 + 0.34) = 1.49$. Better than 0.74, but still not great.

At 90% accuracy: $0.10 \times 10 \times 0.17 = 0.17$ cycles penalty $\Rightarrow$ IPC = 1.71.

At 95% accuracy: $0.05 \times 10 \times 0.17 = 0.085$ cycles penalty $\Rightarrow$ IPC = 1.84.

Every additional point of predictor accuracy is worth real performance.

This is why processor designers spend enormous effort on increasingly sophisticated predictors. Modern CPUs have predictors with $> 95\%$ accuracy on most workloads.

Concept Check 1

1. In the calculation above, we assumed the misprediction penalty is 10 cycles. What does that number represent physically? Why not 5 cycles or 20?
2. Why is a 2-wide superscalar more sensitive to branch misprediction than a single-issue scalar pipeline? Show with arithmetic.
3. A perfect predictor achieves IPC = 2.0 on our 2-wide design. If the predictor is 99% accurate, what IPC do we achieve (using the same assumptions: 17% branches, 10-cycle penalty)?

Going deeper: Hennessy & Patterson §3.3 covers branch prediction with realistic numbers. Onur Mutlu's ETH lectures have an excellent series on branch prediction — watch the 2-bit predictor lecture this week.

3 Comprehensive Theory 2: Static Prediction and 1-Bit Dynamic [~20 min]

3.1 Static Prediction

Static prediction means the prediction is fixed at compile time — the hardware doesn't learn anything at runtime.

Simplest static: “always not taken.” Every branch is predicted not taken; the front-end fetches the next sequential instruction. Accuracy: depends on workload, typically 40–50%. Bad.

Slightly better: “backward-taken, forward-not-taken” (BTFN). If the branch target address is *less than* the current PC (a backward branch), predict taken. Otherwise, predict not taken. The intuition: loops always branch backward, and loops are usually taken (only the last iteration exits). Forward branches are often if-statements; they're harder to predict, so default to not-taken.

BTFN accuracy: ~60%. This is the floor of useful prediction. Worth it for very simple pipelines; not enough for modern OoO.

3.2 Why Static Isn't Enough

Consider a program with two loops, one tight and one rarely-executed:

```
for (i = 0; i < 1000000; i++) {           # Tight loop, branches taken 999999 of 1
    do_work(i);
}

if (rare_condition) {                    # Forward branch, taken < 1% of the tim
    handle_error();
}
```

A static predictor treats every backward branch the same. But the loop branch is taken 99.9999% of the time and the error-check branch is taken < 1% of the time. Static prediction can't distinguish them — it gives the same prediction for both.

The fix: **remember the history of each branch.** This is dynamic prediction.

3.3 The 1-Bit Predictor

The simplest dynamic predictor: for each branch, store one bit recording whether it was taken last time. Predict the same direction next time.

Storage: a table of 1-bit entries, indexed by some hash of the branch PC. Let's say 1024 entries, so the index is 10 bits of the PC.

Update: when the branch resolves, write the actual outcome to the predictor entry. (This is one of the few cases in a pipeline where you write back to a structure based on EX-stage information.)

Accuracy of 1-bit: ~80% on typical code. Much better than static.

3.4 The 1-Bit Predictor's Failure Mode

1-bit prediction has a specific weakness: **single-instance exceptions kill prediction for two iterations.**

Consider a tight loop:

```
for (i = 0; i < 1000; i++) {  
    if (i % 100 == 0) special_case(i);    # Taken once every 100 iterations  
    normal_work(i);  
}  
loop_end:
```

The outer loop branch is taken 999 times then not-taken once. Look at what the 1-bit predictor does on the last iteration:

- Iterations 1–999: predictor says “taken” (matches the last iteration). All correct.
- Iteration 1000: predictor says “taken,” actual is “not taken.” **Mispredict.** Predictor flips to “not taken.”

Fine. The cost is one mispredict per loop execution. But now suppose we enter the loop again later:

- Iteration 1 of the new loop entry: predictor says “not taken” (left over from the last iteration of the previous loop). Actual is “taken.” **Mispredict.** Predictor flips back to “taken.”

So we got *two* mispredictions per loop execution: one at exit, one at re-entry. For a frequently-called loop, this is significant.

The 2-bit predictor solves this.

Concept Check 2

1. Why does BTFN predict backward branches as taken? Give an example of a backward branch that is *not* taken most of the time.
2. In the 1-bit predictor failure-mode example above, how many mispredictions does a single execution of the loop incur (assuming 1000 iterations)? How does this compare to a perfect predictor?
3. How much storage does a 1024-entry 1-bit predictor require, in bits?

4 Comprehensive Theory 3: 2-Bit Counters, BHT, BTB, Speculation [~35 min]

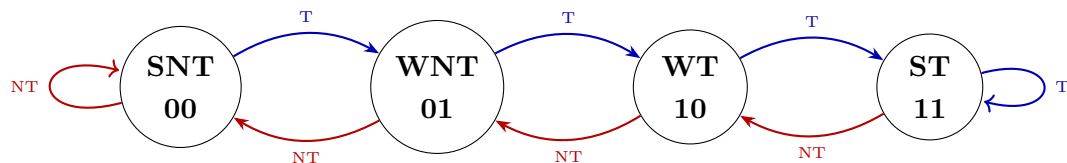
4.1 The 2-Bit Saturating Counter

The 2-bit saturating counter is the canonical dynamic branch predictor. Every branch has a 2-bit state that updates based on its outcome.

The four states:

State	Binary	Meaning
Strongly Not Taken	00	Predict NT; need 2 takens to flip
Weakly Not Taken	01	Predict NT; one taken flips to WT
Weakly Taken	10	Predict T; one not-taken flips to WNT
Strongly Taken	11	Predict T; need 2 not-takens to flip

The state machine:



Key property: a single outlier outcome doesn't flip the prediction. You need *two* consecutive mispredictions to change direction. This is exactly what fixes the 1-bit failure mode.

4.2 Tracing the 2-Bit Predictor on the Earlier Example

Recall the loop that runs 1000 iterations, each time:

- Initial state: SNT (00). First execution will mispredict on iteration 1.
- Iteration 1: predict NT, actual T. Mispredict. State → WNT.
- Iteration 2: predict NT, actual T. Mispredict. State → WT.
- Iteration 3: predict T, actual T. Correct. State → ST.
- Iterations 4–999: predict T, actual T. Correct, state stays ST.
- Iteration 1000 (loop exit): predict T, actual NT. Mispredict. State → WT.

Total mispredictions on first execution: 3.

Second execution of the same loop:

- State entering: WT (from last execution's exit).
- Iteration 1: predict T, actual T. Correct. State → ST.

- Iterations 2–999: correct, state stays ST.
- Iteration 1000: predict T, actual NT. Mispredict. State $\rightarrow$ WT.

Only 1 mispredict per execution after the first. Compare to 1-bit predictor’s 2 per execution. The 2-bit predictor is strictly better and is the universal standard.

4.3 The Branch History Table (BHT)

We can’t have one 2-bit counter per branch in the program — there could be millions of branches. Instead, we use a **Branch History Table**: an array of 2-bit counters, indexed by some hash of the branch’s PC.

Typical size: 1024 to 16384 entries.

Indexing: take low-order bits of the PC. For a 1024-entry table, that’s PC[11:2] (skipping the bottom 2 bits since instructions are 4-byte aligned).

Collisions: two different branches whose PCs happen to hash to the same BHT entry will interfere with each other’s predictions. This is called *aliasing*, and it’s the main source of error in simple predictors. Larger BHTs reduce aliasing.

No tag: the BHT does not check if the entry actually belongs to this branch. We just trust the hash. The prediction is best-effort.

4.4 The Branch Target Buffer (BTB)

The BHT predicts *direction*: taken or not taken. But if a branch is predicted taken, we also need to know **where to fetch from**. That’s the BTB.

The BTB is a small cache that maps PC $\rightarrow$ **target_address**. When the front-end fetches a PC, it consults the BTB:

- BTB hit + BHT says “taken” $\rightarrow$ fetch from BTB target.
- BTB hit + BHT says “not taken” $\rightarrow$ fetch PC+4.
- BTB miss $\rightarrow$ assume not-a-branch, fetch PC+4. (Some designs combine this with static prediction.)

Why we need the BTB: computing the branch target requires decoding the instruction (to extract the offset). If we wait for decode, we’ve already wasted 1 cycle. The BTB caches the answer from previous executions, letting us redirect fetch in the *same cycle* as the prediction.

Typical BTB size: 256 to 2048 entries, fully associative or set-associative.

4.5 Putting Them Together: The Front-End Speculation Loop

1. Fetch reads instruction at current PC. In parallel, BHT and BTB are looked up using PC.

2. BHT returns prediction (T/NT). BTB returns target if hit.
3. Update PC:
 - If BHT says T and BTB hits: $PC \leftarrow \text{BTB target}$.
 - Otherwise: $PC \leftarrow PC + 4$.
4. The instruction proceeds down the pipeline, tagged with the predicted direction and target.
5. In the EX stage, the branch resolves. If prediction was wrong, flush younger pipeline stages (Week 6) and update PC to the correct address. Also update BHT and BTB based on the actual outcome.

4.6 Speculation and the ROB

We covered this in Week 3, but it's worth restating: speculative instructions enter the ROB normally. They allocate physical registers, may even execute and write to the PRF. The *only* difference is that their results are not architecturally visible until commit. When the predicted branch resolves correctly, speculation is silent — those instructions commit normally.

When wrong, the ROB walks from tail back to the mispredicted branch, restoring rename map entries from `old_phys_dest` and returning new physical registers to the free list. Speculation that took 100 cycles to execute can be undone in just a few cycles of cleanup.

4.7 What PRAVAH Will Implement

For Week 6 integration, PRAVAH uses:

- **2-bit saturating counter BHT**, 256 entries (small but sufficient for our small test programs).
- **BTB**: 32 entries, direct-mapped. Optional — if you implement it, accuracy improves; if not, predict-taken branches stall one extra cycle to compute the target.
- Branch resolution in the EX stage (not earlier).
- Single-cycle misprediction penalty for our pipeline depth.

You do not need to implement the predictor this week. You only need to lock down these numbers in your design-decisions document.

Concept Check 3

1. Trace the 2-bit predictor on the sequence T T T T NT T T T T (a loop that runs 4 times, exits once, then runs 4 more times). Starting from SNT, how many mispredictions?
2. Suppose two branches have PCs 0x1000 and 0x5000, and the BHT has 256 entries indexed by PC[9:2]. Do they alias? Show the index computation.
3. Why does the BTB store both PC and target, while the BHT stores only the counter?

Going deeper: McFarling's 1993 paper, "Combining Branch Predictors," introduced gshare and tournament predictors. Modern predictors (TAGE, perceptron) are descendants. For PRAVAH, the 2-bit BHT is enough — but read the McFarling paper if you're curious.

5 Comprehensive Theory 4: The Reservation Station Micro-Architecture [~30 min]

Now we switch tracks. The rest of Week 4 is about the reservation station, which you'll build in Verilog.

5.1 The Mental Model: A Waiting Room

A reservation station is a single slot that holds one instruction waiting to execute. It has:

- A **busy bit**: is this slot occupied?
- An **operation field**: ADD, SUB, AND, etc.
- Two **source physical register indices** (P_j and P_k).
- A **destination physical register index** (P_d).
- A **ROB entry index** (to inform the ROB when the result is computed).

Every cycle, the RS does three things in parallel:

1. **Snoop the PRF ready bits** at addresses P_j and P_k . If both are ready, this RS is *wakeup-ready*.
2. **Raise a wakeup-ready signal** to the issue logic.
3. **If selected to issue this cycle**: read operand values from $\text{PRF}[P_j]$ and $\text{PRF}[P_k]$, send the operation and operands to the FU, and clear the busy bit (the slot becomes available for new dispatches next cycle).

That's it. The RS is genuinely simple. The complexity is in the *array* of RSs operating in parallel.

The reservation station does not wait for execution to complete; it clears its busy bit on the exact same clock cycle it issues the instruction to the functional unit. The reservation station acts purely as a temporary staging queue or "waiting room" to handle data dependencies. The moment an instruction's operands become ready and it wins arbitration via the priority encoder, its entire metadata payload (opcode, source indices, and destination tags) is driven onto the combinational wires and immediately swallowed by the input registers of the execution unit. Because the functional unit now fully owns the instruction and tracks its multi-cycle computation through its own internal pipeline registers, keeping the reservation station slot locked would pointlessly waste a slot and cause structural stalls for younger independent instructions. Therefore, on the very next rising clock edge, `rs_busy` is safely flipped back to zero, making that exact slot immediately vacant and capable of accepting a brand-new dispatched instruction while the older instruction continues executing down the backend pipeline

5.2 The Reservation Station Array

We have multiple RSs attached to each functional unit. For PRAVAH:

- 4 RSs attached to the ALU pool (2 ALUs share them).
- 2 RSs attached to the multiplier.
- 2 RSs attached to the load/store unit.

This week we'll build the **4-entry ALU RS array** only. The MUL and LSU RS arrays use the same structure with different parameters — you'll instantiate them in Week 6.

5.3 The Three Phases of an RS Cycle

Let's trace what happens to an RS array in one clock cycle:

Phase 1 (early in cycle): wakeup. Every busy RS reads the PRF ready bits for its two source physical registers. If both are ready, the RS sets its internal `wakeup_ready` signal.

Phase 2 (mid-cycle): select. A priority encoder examines all `wakeup_ready` signals. It picks one (typically the lowest-indexed) and asserts the issue signal for that RS. *In future weeks we'll have multiple ALUs — the priority encoder picks one per ALU.*

Phase 3 (late in cycle): issue. The selected RS sends its operation and source physical register indices to the ALU. The ALU reads operands from $\text{PRF}[P_j]$ and $\text{PRF}[P_k]$ (these reads are combinational, since the ALU has read ports on the PRF). The ALU's combinational logic begins computing. At the end of the cycle, the RS clears its busy bit.

Phase 4 (next cycle and beyond): execute and complete. The ALU completes its operation in some number of cycles (1 for single-cycle ALU; multiple for MUL). When done, it writes the result to $\text{PRF}[P_d]$, sets $\text{PRF}[P_d].\text{ready}$, and signals the ROB to mark its entry ready.

5.4 A Worked Cycle-by-Cycle Trace

Setup: 4-entry ALU RS array. Initially empty. PRF has 48 entries, all ready initially (since they haven't been allocated).

Code:

```
I1: add x5, x1, x2      # renamed to: add P32, P1, P2
I2: add x6, x5, x3      # renamed to: add P33, P32, P3 (depends on I1)
I3: sub x7, x1, x4      # renamed to: sub P34, P1, P4 (independent)
```

When I1 is dispatched, P32's ready bit is cleared (it's the new destination, value not yet computed). I2's dispatch clears P33's ready. I3's clears P34's.

Cycle T: Dispatch I1 into RS[0]. RS[0]: {busy=1, op=ADD, P_j =P1, P_k =P2, P_d =P32, rob_idx=0}.

Wakeup phase: RS[0] reads PRF[P1].ready=1 and PRF[P2].ready=1. Both ready. `wakeup_ready[0] = 1`.

Select phase: only RS[0] is wakeup-ready. Issue.

Issue phase: ALU reads PRF[P1] and PRF[P2], begins computing. `RS[0].busy ← 0`.

Cycle T+1: Dispatch I2 into RS[1] (RS[0] is now free). I1 finishes executing.

At start of cycle T+1: I1's ALU result is written to PRF[P32], `PRF[P32].ready ← 1`.

I2 dispatch: RS[1]: {busy=1, op=ADD, $P_j=P32$, $P_k=P3$, $P_d=P33$, `rob_idx=1`}.

Wakeup phase: RS[1] reads PRF[P32].ready=1 (just set!) and PRF[P3].ready=1. Both ready. `wakeup_ready[1] = 1`.

This is the magic of the design. I2 was just dispatched, and in the same cycle, its operand is already ready. No waiting. The wakeup logic instantly captures the PRF state.

Issue phase: RS[1] issues to the ALU.

Cycle T+1 (continued): Also dispatch I3 into RS[2].

In a 2-wide dispatch (Week 5), we dispatch both I2 and I3 in the same cycle. In Week 4's single-issue version, we dispatch only one per cycle. Let's say I3 dispatches in cycle T+2.

Cycle T+2: I3 in RS[2].

RS[2]: {busy=1, op=SUB, $P_j=P1$, $P_k=P4$, $P_d=P34$, `rob_idx=2`}.

Wakeup: PRF[P1].ready=1 and PRF[P4].ready=1. `wakeup_ready[2] = 1`.

Issue.

Key observations:

- I2 was issued one cycle after dispatch despite depending on I1, because I1 finished in time.
- I3 issued the cycle of its dispatch since both its operands were always ready.
- Out-of-order execution would manifest if I2 were a slow MUL — then I3 (in RS[2]) would issue *before* I2's result was ready, even though I2 was dispatched first.
- Cycle T : I1 is in RS[0]. It wakes up and issues to the ALU. The hardware schedules `rs_busy[0]` to clear. Clock Tick ($\uparrow$): `rs_busy[0]` transitions from 1 to 0. Cycle $T + 1$: RS[0] is now officially empty. At the exact same time, I2 is dispatching. Because the dispatch logic evaluates the state arrays simultaneously, I2 defaults into RS[1] because RS[0] was only just cleared at the boundary edge. If you wanted I2 to instantly reuse RS[0] in the same cycle it was cleared, you would have to build an explicit dispatch-bypass multiplexer network. "We don't add this for Week 4; it's a minor performance optimization, not a correctness issue."

5.5 What Happens When Multiple RSs Are Wakeup-Ready?

This is the case the priority encoder handles. Suppose RS[0], RS[1], and RS[3] are all wakeup-ready in the same cycle. We have only 1 ALU (for simplicity — PRAVAH has 2). The select logic must pick exactly one.

Simple strategy: lowest-index priority. RS[0] wins. RS[1] and RS[3] wait one more cycle.

Better strategy: oldest-first. Track the dispatch cycle of each RS and pick the oldest. Slightly more expensive but reduces certain stalls. PRAVAH will use **lowest-index priority** for simplicity — it's adequate for our workloads.

5.6 What the RS Does NOT Do

A few things to be clear about:

- The RS **does not store operand values**. It stores physical register indices. The actual values live in the PRF.
- The RS **does not compute**. It just waits for sources to be ready and then hands off to the FU.
- The RS **is not part of the ROB**. It tracks a ROB index (so the ROB can be informed when the result is computed) but is structurally independent.
- The RS **does not handle branch recovery directly**. On flush, the RS array is cleared by an external signal (Week 6). For Week 4, we won't implement flush; that comes with full integration.

Concept Check 4

1. In the worked example, I2 depends on I1. We saw I2 issue one cycle after dispatch. Suppose I1 took 4 cycles instead of 1 to execute. When does I2 issue?
2. Why does the RS store only *indices* into the PRF, not the operand values themselves? What would change if it stored values?
3. In Phase 1 (wakeup), every busy RS reads the PRF ready bits. For a 4-entry RS array, that's $4 \text{ RSs} \times 2 \text{ source reads} = 8$ simultaneous reads of the ready bits. Where do those 8 read ports go? (Hint: think about how the PRF was designed in Week 3.)

Going deeper: Shen & Lipasti Chapter 5 has the cleanest treatment of RS implementation. Onur Mutlu's lectures on dynamic scheduling cover wakeup-select in depth.

6 Module: reservation_station.v [~1 hour]

This week's Verilog module is the ALU reservation station array. It's structurally similar to Week 3's modules but has more interesting control logic.

6.1 Specification

- 4 entries, each holding one ALU instruction.
- Each entry tracks: busy, op, P_j , P_k , P_d , rob_idx.
- Single-cycle dispatch port (writes one new entry per cycle).
- Wakeup logic that snoops PRF ready bits for all 4 entries in parallel.
- Lowest-index priority encoder for select.
- Issue port that drives the ALU with op, P_j , P_k index, P_d index, rob_idx.

6.2 Port List

```
1 module reservation_station #(
2     parameter NUM_RS      = 4,
3     parameter PHYS_ADDR_W = 6,    // log2(48)
4     parameter ROB_ADDR_W  = 3,    // log2(8 ROB entries)
5     parameter OP_WIDTH    = 4     // op encoding bits
6 ) (
7     input  wire          clk_i,
8     input  wire          reset_i,
9
10    // Dispatch port (allocate one RS this cycle)
11    input  wire          disp_valid_i,
12    input  wire [OP_WIDTH-1:0] disp_op_i,
13    input  wire [PHYS_ADDR_W-1:0] disp_pj_i,
14    input  wire [PHYS_ADDR_W-1:0] disp_pk_i,
15    input  wire [PHYS_ADDR_W-1:0] disp_pd_i,
16    input  wire [ROB_ADDR_W-1:0] disp_rob_idx_i,
17    output wire          disp_stall_o, // 1 if no
18    // free RS
19
20    // PRF ready-bit snoop (one per source per RS = 2*NUM_RS
21    // reads)
22    // We expose source addresses as outputs; PRF returns ready
23    // bits.
24    output wire [PHYS_ADDR_W-1:0] snoop_pj_addr_o [0:NUM_RS
25    -1],
```

```

22     output wire [PHYS_ADDR_W-1:0]      snoop_pk_addr_o [0:NUM_RS
        -1],
23     input  wire                        snoop_pj_ready_i [0:NUM_RS
        -1],
24     input  wire                        snoop_pk_ready_i [0:NUM_RS
        -1],
25
26     // Issue port (one issue per cycle to the ALU)
27     output wire                        issue_valid_o ,
28     output wire [OP_WIDTH-1:0]         issue_op_o ,
29     output wire [PHYS_ADDR_W-1:0]      issue_pj_o ,
30     output wire [PHYS_ADDR_W-1:0]      issue_pk_o ,
31     output wire [PHYS_ADDR_W-1:0]      issue_pd_o ,
32     output wire [ROB_ADDR_W-1:0]       issue_rob_idx_o
33 );

```

Note on port arrays: Verilog-2001 doesn't natively support unpacked array ports the way SystemVerilog does. The notation `wire [W-1:0] foo [0:N-1]` for a port works in many simulators and synthesizers, but for maximum portability, you may need to flatten to a single wide bus. For ModelSim-Intel Starter and Quartus, the array notation above should work; if not, declare as a single bus of width `NUM_RS * PHYS_ADDR_W` and slice in the testbench.

6.3 Internal Structure

```

1     // Per-RS state arrays
2     reg                                rs_busy      [0:NUM_RS-1];
3     reg [OP_WIDTH-1:0]                 rs_op        [0:NUM_RS-1];
4     reg [PHYS_ADDR_W-1:0]              rs_pj        [0:NUM_RS-1];
5     reg [PHYS_ADDR_W-1:0]              rs_pk        [0:NUM_RS-1];
6     reg [PHYS_ADDR_W-1:0]              rs_pd        [0:NUM_RS-1];
7     reg [ROB_ADDR_W-1:0]               rs_rob_idx   [0:NUM_RS-1];
8
9     // Combinational signals
10    wire [NUM_RS-1:0] rs_wakeup_ready;    // one bit per RS
11    wire [NUM_RS-1:0] rs_free;            // one bit per RS (1 =
        available for dispatch)
12    wire [NUM_RS-1:0] issue_select;       // one-hot: which RS is
        issuing this cycle
13    wire [NUM_RS-1:0] dispatch_select;    // one-hot: which RS is
        being allocated
14
15    integer i;

```

6.4 Implementation

Step 1: Expose snoop addresses combinationally.

```
1  genvar g;
2  generate
3      for (g = 0; g < NUM_RS; g = g + 1) begin : snoop_gen
4          assign snoop_pj_addr_o[g] = rs_pj[g];
5          assign snoop_pk_addr_o[g] = rs_pk[g];
6      end
7  endgenerate
```

The PRF receives these addresses combinationally and returns `snoop_pj_ready_i[g]` and `snoop_pk_ready_i[g]` in the same cycle.

Step 2: Compute wakeup-ready for each RS.

```
1  generate
2      for (g = 0; g < NUM_RS; g = g + 1) begin : wakeup_gen
3          assign rs_wakeup_ready[g] = rs_busy[g] &
4                                     snoop_pj_ready_i[g] &
5                                     snoop_pk_ready_i[g];
6      end
7  endgenerate
```

An RS is wakeup-ready iff it's busy and both its sources are ready. (A non-busy RS has garbage source indices; we mask it with the busy bit so it doesn't accidentally raise wakeup.)

Step 3: Compute free signals.

```
1  generate
2      for (g = 0; g < NUM_RS; g = g + 1) begin : free_gen
3          assign rs_free[g] = ~rs_busy[g];
4      end
5  endgenerate
6
7  // Stall dispatch if no RS is free
8  assign disp_stall_o = disp_valid_i & (rs_free == {NUM_RS{1'b0}});
```

Step 4: Priority encoders for issue-select and dispatch-select.

For `NUM_RS = 4`, the lowest-index priority encoder is:

```
1  // Issue select: lowest-index wakeup-ready RS wins
2  assign issue_select[0] = rs_wakeup_ready[0];
3  assign issue_select[1] = rs_wakeup_ready[1] & ~
4                          rs_wakeup_ready[0];
5  assign issue_select[2] = rs_wakeup_ready[2] & ~
6                          rs_wakeup_ready[1];
```

```

5                                     & ~
                                     rs_wakeup_ready
                                     [0];
6  assign issue_select[3] = rs_wakeup_ready[3] & ~
    rs_wakeup_ready[2]
7                                     & ~
                                     rs_wakeup_ready
                                     [1]
8                                     & ~
                                     rs_wakeup_ready
                                     [0];
9
10 // Dispatch select: lowest-index free RS wins
11 assign dispatch_select[0] = rs_free[0] & disp_valid_i;
12 assign dispatch_select[1] = rs_free[1] & ~rs_free[0] &
    disp_valid_i;
13 assign dispatch_select[2] = rs_free[2] & ~rs_free[1] & ~
    rs_free[0] & disp_valid_i;
14 assign dispatch_select[3] = rs_free[3] & ~rs_free[2] & ~
    rs_free[1]
15                                     & ~rs_free[0] & disp_valid_i;

```

These are at most 4-input AND gates — cheap.

Step 5: Issue port outputs.

The selected RS drives the issue port outputs. Since `issue_select` is one-hot, we OR-reduce across all RSs:

```

1  assign issue_valid_o = |issue_select;    // any RS issuing?
2
3  // Each output is a multiplexer driven by issue_select
4  assign issue_op_o      = issue_select[0] ? rs_op[0]      :
5                          issue_select[1] ? rs_op[1]      :
6                          issue_select[2] ? rs_op[2]      :
7                          issue_select[3] ? rs_op[3]      : {
9                          OP_WIDTH{1'b0}};
8
9  assign issue_pj_o      = issue_select[0] ? rs_pj[0]      :
10                          issue_select[1] ? rs_pj[1]      :
11                          issue_select[2] ? rs_pj[2]      :
12                          issue_select[3] ? rs_pj[3]      : {
13                          PHYS_ADDR_W{1'b0}};
14
15 assign issue_pk_o      = issue_select[0] ? rs_pk[0]      :
16                          issue_select[1] ? rs_pk[1]      :
17                          issue_select[2] ? rs_pk[2]      :

```

```

17         issue_select[3] ? rs_pk[3]      : {
18             PHYS_ADDR_W{1'b0}};
19
20 assign issue_pd_o      = issue_select[0] ? rs_pd[0]      :
21             issue_select[1] ? rs_pd[1]      :
22             issue_select[2] ? rs_pd[2]      :
23             issue_select[3] ? rs_pd[3]      : {
24                 PHYS_ADDR_W{1'b0}};
25
26 assign issue_rob_idx_o = issue_select[0] ? rs_rob_idx[0] :
27             issue_select[1] ? rs_rob_idx[1] :
28             issue_select[2] ? rs_rob_idx[2] :
29             issue_select[3] ? rs_rob_idx[3] : {
30                 ROB_ADDR_W{1'b0}};

```

Step 6: Synchronous state update.

```

1  always @(posedge clk_i) begin
2      if (reset_i) begin
3          for (i = 0; i < NUM_RS; i = i + 1) begin
4              rs_busy[i]      <= 1'b0;
5              rs_op[i]        <= {OP_WIDTH{1'b0}};
6              rs_pj[i]        <= {PHYS_ADDR_W{1'b0}};
7              rs_pk[i]        <= {PHYS_ADDR_W{1'b0}};
8              rs_pd[i]        <= {PHYS_ADDR_W{1'b0}};
9              rs_rob_idx[i]   <= {ROB_ADDR_W{1'b0}};
10         end
11     end else begin
12         // Issue: clear the busy bit of the issued RS
13         for (i = 0; i < NUM_RS; i = i + 1) begin
14             if (issue_select[i])
15                 rs_busy[i] <= 1'b0;
16         end
17
18         // Dispatch: write into the dispatched RS
19         for (i = 0; i < NUM_RS; i = i + 1) begin
20             if (dispatch_select[i]) begin
21                 rs_busy[i]      <= 1'b1;
22                 rs_op[i]        <= disp_op_i;
23                 rs_pj[i]        <= disp_pj_i;
24                 rs_pk[i]        <= disp_pk_i;
25                 rs_pd[i]        <= disp_pd_i;
26                 rs_rob_idx[i]   <= disp_rob_idx_i;
27             end
28         end
29     end

```



```

30     end
31
32 endmodule

```

Subtle point: can the same RS be issued and dispatched in the same cycle? Yes! Suppose RS[0] is wakeup-ready (issuing this cycle) and the front-end wants to dispatch a new instruction. After the issue, RS[0] is free. The dispatch select logic looks at `rs_free`, which is currently `~rs_busy`. But `rs_busy[0]` is still 1 (it doesn't go to 0 until the clock edge). So dispatch goes to RS[1] instead. After the clock edge: RS[0] is now free (issue cleared it), RS[1] is now busy (dispatch wrote it). This is correct.

If you want dispatch to use RS[0] in the same cycle as its issue, you'd need bypass logic. We don't add this for Week 4; it's a minor performance optimization, not a correctness issue.

6.5 Common Bugs to Watch For

Issue-on-non-busy-RS. If your wakeup logic forgets the `rs_busy[g] &` term, a freshly-reset RS with all-zero source indices may have PRF ready bits set (since P0 is initially ready in the reset state). It would falsely raise wakeup. The busy-bit gating is essential.

Priority encoder forgetting a level. Verify by hand that for any pattern of `wakeup_ready`, exactly one bit of `issue_select` is asserted (or zero, if no RS is ready). A common bug: `issue_select[3]` forgets to AND with `~rs_wakeup_ready[0]`.

Forgetting reset to clear busy bits. If `rs_busy[i]` comes up as X (unknown) at simulation start, your wakeup logic propagates Xs everywhere. Always explicitly reset all RS state.

Dispatch and issue racing on the same RS. The subtle point above: in the same cycle, an RS that's about to issue is still marked busy when dispatch looks. So a new dispatch can't take its slot until the next cycle. This is correct behavior; just don't "fix" it by mistake.

Concept Check 5

1. Why does the wakeup-ready signal include `rs_busy[g] &`? What would happen without it?
2. In the issue-output multiplexer, each output is selected from 4 sources by a 4:1 mux. How many gates does this mux take, approximately?
3. Suppose 3 RSs are wakeup-ready in the same cycle. Which one issues? Which RSs do not issue and what happens to them next cycle?

7 Testbench: `tb_reservation_station.v` [~45 min]

The RS testbench is more interesting than Week 3's. We need to model the surrounding behavior (PRF ready bits, dispatch port, issue consumer) and check that the RS responds correctly.

7.1 What the Testbench Must Cover

1. Reset clears all RS busy bits.
2. Dispatch into RS[0] when empty: RS[0] becomes busy.
3. Dispatch with ready sources: same-cycle wakeup and issue next cycle.
4. Dispatch with one not-ready source: RS stays busy until that source becomes ready.
5. Multiple dispatches fill RS[0], RS[1], etc. in lowest-index-free order.
6. Multiple wakeup-ready RSs in same cycle: lowest index issues; others wait.
7. Full RS array: dispatch is stalled.
8. Reset clears mid-execution state.
9. RS[0] issues, freeing its slot; next dispatch correctly fills RS[0] again.

7.2 Testbench Skeleton

```
1  `timescale 1ns / 1ps
2
3  module tb_reservation_station();
4
5      // Parameters
6      localparam NUM_RS      = 4;
7      localparam PHYS_ADDR_W = 6;
8      localparam ROB_ADDR_W  = 3;
9      localparam OP_WIDTH    = 4;
10
11     // Clock & reset
12     reg clk = 0;
13     reg reset = 1;
14     always #5 clk = ~clk;
15
16     // Dispatch port
17     reg                                disp_valid;
18     reg [OP_WIDTH-1:0]                disp_op;
19     reg [PHYS_ADDR_W-1:0]             disp_pj;
```

```

20  reg [PHYS_ADDR_W-1:0]  disp_pk;
21  reg [PHYS_ADDR_W-1:0]  disp_pd;
22  reg [ROB_ADDR_W-1:0]   disp_rob_idx;
23  wire                    disp_stall;
24
25  // Snoop ports (we drive these from a fake PRF ready-bit
   model)
26  wire [PHYS_ADDR_W-1:0] snoop_pj_addr [0:NUM_RS-1];
27  wire [PHYS_ADDR_W-1:0] snoop_pk_addr [0:NUM_RS-1];
28  reg                    snoop_pj_ready [0:NUM_RS-1];
29  reg                    snoop_pk_ready [0:NUM_RS-1];
30
31  // Issue port (we capture)
32  wire                    issue_valid;
33  wire [OP_WIDTH-1:0]    issue_op;
34  wire [PHYS_ADDR_W-1:0] issue_pj;
35  wire [PHYS_ADDR_W-1:0] issue_pk;
36  wire [PHYS_ADDR_W-1:0] issue_pd;
37  wire [ROB_ADDR_W-1:0]  issue_rob_idx;
38
39  // Fake PRF: track which physical registers are ready
40  reg [47:0] prf_ready_bits; // 48 phys regs, 1 bit each
41
42  // Drive snoop ready inputs based on fake PRF state
43  integer s;
44  always @* begin
45      for (s = 0; s < NUM_RS; s = s + 1) begin
46          snoop_pj_ready[s] = prf_ready_bits[snoop_pj_addr[s]];
47          snoop_pk_ready[s] = prf_ready_bits[snoop_pk_addr[s]];
48      end
49  end
50
51  // Instantiate DUT
52  reservation_station dut (
53      .clk_i(clk), .reset_i(reset),
54      .disp_valid_i(disp_valid), .disp_op_i(disp_op),
55      .disp_pj_i(disp_pj), .disp_pk_i(disp_pk), .disp_pd_i(
56          disp_pd),
57      .disp_rob_idx_i(disp_rob_idx), .disp_stall_o(disp_stall),
58      .snoop_pj_addr_o(snoop_pj_addr), .snoop_pk_addr_o(
59          snoop_pk_addr),
58      .snoop_pj_ready_i(snoop_pj_ready), .snoop_pk_ready_i(
59          snoop_pk_ready),
59      .issue_valid_o(issue_valid), .issue_op_o(issue_op),

```

```

60     .issue_pj_o(issue_pj), .issue_pk_o(issue_pk), .issue_pd_o
        (issue_pd),
61     .issue_rob_idx_o(issue_rob_idx)
62 );
63
64 // Self-checking helper
65 integer errors = 0;
66 task check_eq;
67     input [31:0] actual;
68     input [31:0] expected;
69     input [255:0] label;
70     begin
71         if (actual === expected)
72             $display("PASS: %s (got %0d)", label, actual);
73         else begin
74             $display("FAIL: %s (expected %0d, got %0d)",
75                 label, expected, actual);
76             errors = errors + 1;
77         end
78     end
79 endtask
80
81 // Test sequence
82 initial begin
83     // Init
84     disp_valid = 0; disp_op = 0;
85     disp_pj = 0; disp_pk = 0; disp_pd = 0; disp_rob_idx = 0;
86     prf_ready_bits = 48'hFFFF_FFFF_FFFF; // all ready
87     initially
88     #20; reset = 0;
89
90     // ----- Test 1: Dispatch one ready instruction -----
91     // P1 and P2 are ready, P32 (dest) initially ready (will
92     be cleared by RAT in real design;
93     // here we just test the RS logic in isolation).
94     @(posedge clk);
95     disp_valid <= 1;
96     disp_op <= 4'd1; // ADD
97     disp_pj <= 6'd1;
98     disp_pk <= 6'd2;
99     disp_pd <= 6'd32;
100    disp_rob_idx <= 3'd0;
101    @(posedge clk);
102    disp_valid <= 0;
103    // Check that the instruction issues

```

```

101     #1;
102     check_eq(issue_valid, 1'b1, "T1: issue_valid after
        dispatch");
103     check_eq(issue_op, 4'd1, "T1: issue_op = ADD");
104     check_eq(issue_pj, 6'd1, "T1: issue_pj = P1");
105     check_eq(issue_pk, 6'd2, "T1: issue_pk = P2");
106     check_eq(issue_pd, 6'd32, "T1: issue_pd = P32");
107
108     // ----- Test 2: Dispatch with a not-ready source -----
109     // Mark P5 as not ready
110     prf_ready_bits[5] = 1'b0;
111     @(posedge clk);
112     disp_valid <= 1;
113     disp_op <= 4'd2;      // SUB
114     disp_pj <= 6'd5;      // not ready
115     disp_pk <= 6'd6;      // ready
116     disp_pd <= 6'd33;
117     disp_rob_idx <= 3'd1;
118     @(posedge clk);
119     disp_valid <= 0;
120     #1;
121     check_eq(issue_valid, 1'b0, "T2: issue_valid = 0 (waiting
        for P5)");
122
123     // Now make P5 ready
124     prf_ready_bits[5] = 1'b1;
125     #1;
126     check_eq(issue_valid, 1'b1, "T2: issue_valid = 1 after P5
        ready");
127     check_eq(issue_pj, 6'd5, "T2: issue_pj = P5");
128     @(posedge clk);
129     #1;
130
131     // ----- Test 3: Fill all 4 RSs (some with not-ready
        sources) -----
132     // Reset state: all RSs free
133     // Mark P10, P11, P12, P13 as not ready
134     prf_ready_bits[10] = 1'b0;
135     prf_ready_bits[11] = 1'b0;
136     prf_ready_bits[12] = 1'b0;
137     prf_ready_bits[13] = 1'b0;
138
139     // Dispatch 4 instructions, each waiting on one of the
        not-ready regs
140     @(posedge clk); disp_valid <= 1;

```

```

141     disp_op <= 4'd1; disp_pj <= 6'd10; disp_pk <= 6'd2;
142         disp_pd <= 6'd34; disp_rob_idx <= 3'd2;
143     @(posedge clk);
144     disp_op <= 4'd1; disp_pj <= 6'd11; disp_pk <= 6'd2;
145         disp_pd <= 6'd35; disp_rob_idx <= 3'd3;
146     @(posedge clk);
147     disp_op <= 4'd1; disp_pj <= 6'd12; disp_pk <= 6'd2;
148         disp_pd <= 6'd36; disp_rob_idx <= 3'd4;
149     @(posedge clk);
150     disp_op <= 4'd1; disp_pj <= 6'd13; disp_pk <= 6'd2;
151         disp_pd <= 6'd37; disp_rob_idx <= 3'd5;
152     @(posedge clk);
153     disp_valid <= 0;
154     #1;
155     check_eq(issue_valid, 1'b0, "T3: no issue (all waiting)")
156         ;
157
158     // Try to dispatch a 5th -- should stall
159     @(posedge clk);
160     disp_valid <= 1;
161     disp_op <= 4'd1; disp_pj <= 6'd2; disp_pk <= 6'd2;
162         disp_pd <= 6'd38; disp_rob_idx <= 3'd6;
163     #1;
164     check_eq(disp_stall, 1'b1, "T3: disp_stall when array
165         full");
166     @(posedge clk);
167     disp_valid <= 0;
168
169     // Now make P12 ready -- RS[2] should issue
170     prf_ready_bits[12] = 1'b1;
171     #1;
172     check_eq(issue_valid, 1'b1, "T3: issue after P12 ready");
173     check_eq(issue_pj, 6'd12, "T3: P12 issued first");
174
175     // ----- Test 4: Multiple ready in same cycle -----
176     // Now make P10, P11, P13 ready too. All three remaining
177     // RSs are wakeup-ready.
178     prf_ready_bits[10] = 1'b1;
179     prf_ready_bits[11] = 1'b1;
180     prf_ready_bits[13] = 1'b1;
181     @(posedge clk); // issue the previously-ready one
182     #1;
183     check_eq(issue_valid, 1'b1, "T4: lowest-index wins (P10)"
184         );

```

```

176         check_eq(issue_pj, 6'd10, "T4: P10 issued (lowest free
           index)");
177
178         // Continue issuing one per cycle
179         @(posedge clk);
180         #1;
181         check_eq(issue_pj, 6'd11, "T4: P11 issued next");
182         @(posedge clk);
183         #1;
184         check_eq(issue_pj, 6'd13, "T4: P13 issued last");
185
186         @(posedge clk);
187         #1;
188         check_eq(issue_valid, 1'b0, "T4: all issued, no more");
189
190         // ----- Done -----
191         if (errors == 0)
192             $display("ALL TESTS PASSED (%0d errors)", errors);
193         else
194             $display("FAILED %0d TESTS", errors);
195         $finish;
196     end
197
198 endmodule

```

7.3 Running the Testbench

```

cd sim/
vlib work
vlog ../rtl/reservation_station.v ../tb/tb_reservation_station.v
vsim -c -do "run -all; quit" tb_reservation_station

```

If using ModelSim GUI to inspect waveforms:

```

vsim tb_reservation_station
add wave -recursive *
run -all

```

Key signals to watch in the waveform:

- `rs_busy[]` — which RSs are occupied.
- `rs_wakeup_ready[]` — which RSs have ready operands.
- `issue_select[]` — which RS is issuing (should be one-hot).
- `issue_valid_o` and `issue_pj_o`, `issue_pk_o`, `issue_pd_o`.

Take screenshots of waveforms for Test 3 (full array) and Test 4 (multiple-wakeup priority) for the deliverable.

Concept Check 6

1. In Test 2, why is the check `issue_valid = 1` valid #1 after making P5 ready, without a clock edge? (Hint: think about whether wakeup logic is combinational or sequential.)
2. In Test 4, three instructions are wakeup-ready at the same cycle. Why does only one issue per cycle, and not all three at once? What would we need to change to allow multiple issues?
3. Why does the testbench model the PRF as a register array (`prf_ready_bits`) rather than instantiating the actual Week 3 `register_file` module?

8 The Design Decisions Document [~1 hour]

This is the second half of Milestone 2. You must produce a one-page document (`docs/design_decisions`) that locks down every architectural parameter PRAVAH will use. The point is to force you to commit *now* so we don't change scope later.

8.1 Required Content

Your document must answer every one of the following questions with a specific number or short phrase. “We’ll figure it out later” is not an acceptable answer for any of them.

8.1.1 Pipeline width

- Issue width: 2 (fixed for PRAVAH).
- Fetch width: 2 (matches issue, simplifies dispatch).
- Commit width: 2 (so we don't accumulate uncommitted instructions during good runs).

8.1.2 Register file sizing

- Architectural registers: 32 (RV32I).
- Physical registers: 48.
- Free list size: 16 ($= 48 - 32$).

8.1.3 Reservation stations

- ALU RSs: 4.
- Multiplier RSs: 2.
- Load/Store RSs: 2.
- Total: 8 (matches typical small-machine ratios).

8.1.4 Reorder buffer

- Size: 8 entries (small but enough for visible OoO).
- Width: 2 dispatch/commit per cycle.

8.1.5 Functional units

- ALUs: 2 (single-cycle latency).
- Multiplier: 1 (3-cycle pipelined).
- Load/Store unit: 1 (1-cycle for hits, blocking).

8.1.6 Branch predictor

- Type: 2-bit saturating counter BHT.
- BHT entries: 256.
- BTB: 32 entries, direct-mapped (or omit BTB for simplicity — explicitly say so).
- Resolution stage: EX.

8.1.7 ISA subset

PRAVAH supports the following RV32I instructions (lock this down now — adding more later costs major effort):

- Arithmetic: ADD, SUB, ADDI.
- Logical: AND, OR, XOR.
- Shift: SLL, SRL.
- Compare: SLT.
- Memory: LW, SW.
- Branch: BEQ, BNE.
- Jump: JAL.

That's 13 instructions, comfortably within scope.

8.1.8 Memory model

- Instruction memory: 1 KB, initialized via `$readmemh` in testbench.
- Data memory: 1 KB, single-port.
- Both treated as combinational reads (no cache modeling).

8.1.9 What we are NOT building

Explicitly list non-features so scope stays clean:

- No cache hierarchy (memories are flat).
- No load/store queue (loads block).
- No store-to-load forwarding.
- No tournament or gshare predictor (just 2-bit BHT).
- No floating-point (RV32I only).

- No interrupts or exceptions (assume well-formed input programs).
- No 4-wide issue.

8.2 Format of the Document

A single markdown file, ~1 page (or 60–90 lines). Sections for each of the above bullet groups. The document is a **contract**: everything in it is committed for Weeks 5–8.

8.3 Why This Matters

In Week 6, when you’re 50 hours into integration debugging, you will be tempted to change a parameter (“maybe 16 ROB entries would be better”) to avoid fixing a real bug. The design-decisions document is the artifact that says “no, the bug is real, fix the bug.” Without it, mentees endlessly tweak parameters chasing imaginary fixes.

The document also gives mentor and mentee a shared reference. When something breaks, we both look at the document first. If the implementation diverges from the document, we either fix the implementation or update the document (with a written reason). No silent drift.

8.4 Example Stub

```
# PRAVAH Design Decisions

Locked down at Week 4, Milestone 2. Changes require mentor approval.

## Pipeline
- Issue width: 2
- Fetch width: 2
- Commit width: 2

## Register file
- Architectural regs: 32 (RV32I)
- Physical regs: 48
- Free list: 16 entries

## Reservation stations
- ALU: 4 entries (this week’s module)
- MUL: 2 entries (Week 6)
- LSU: 2 entries (Week 6)

## ROB
- Depth: 8
- Dispatch/commit width: 2
```

```

## Functional units
- 2 ALUs (1-cycle)
- 1 MUL (3-cycle pipelined)
- 1 LSU (1-cycle, blocking)

## Branch predictor
- 2-bit BHT, 256 entries, indexed by PC[9:2]
- No BTB (predict-taken pays 1 extra cycle for target computation)

## ISA subset (13 instructions)
- ADD, SUB, ADDI
- AND, OR, XOR, SLL, SRL, SLT
- LW, SW
- BEQ, BNE, JAL

## Memory
- IMem: 1 KB, $readmemh init
- DMem: 1 KB, single-port, combinational read

## Out of scope (DO NOT IMPLEMENT)
- Caches, LSQ, store forwarding, tournament predictor, FP, exceptions, 4-wid

```

9 Week 4 Deliverables — Milestone 2

In Your Repo

- `rtl/reservation_station.v` — 4-entry ALU RS array per spec in Section 5.
- `tb/tb_reservation_station.v` — self-checking testbench covering all 4 tests in Section 6.
- `docs/design_decisions.md` — 1-page locked architectural spec per Section 7.
- `sim/waveforms/rs_test3.png` and `rs_test4.png` — ModelSim screenshots.
- Updated `learning_log.md`.

What Mentor Will Check (Milestone 2 Gate)

1. **Compiles in Quartus** without errors.
2. **All testbench cases pass** (ALL TESTS PASSED in transcript).
3. **Wakeup-ready logic includes the busy-bit gate**, no false wakeups.
4. **Priority encoder is correct** — spot-check by hand with a few wakeup patterns.
5. **Design-decisions document is complete** — every parameter has a number, no “TBD”.
6. **ISA subset matches the document** — if you’ve added or removed instructions silently, that’s a fail.

Common Bugs to Watch For

- **Forgetting the busy-bit gate** in wakeup-ready: causes false issues from freshly-reset RSs.
- **Priority encoder asymmetry** between `issue_select` and `dispatch_select`: it’s easy to write one correctly and mess up the other.
- **Issue mux missing a default value**: if no RS issues, the `issue_pj/pk/pd` outputs should not be X — assign zeros or some defined value.
- **Snoop port wiring**: the `snoop_pj/pk_addr` outputs must be the *current* source indices, not stale ones. Use combinational assigns.

Time Estimate

- Theory (Sections 1–3, branch prediction): 2–2.5 hours.
- Theory (Section 4, RS micro-architecture): 45 min–1 hour.
- Verilog reading (Sections 5–6): 1–1.5 hours.
- Coding RS + testbench: 4–5 hours.
- Writing design-decisions document: 1 hour.
- Debugging: 1–2 hours.

If you finish early: take an hour to read about gshare or tournament predictors. They'll come up in Week 6 if you want a stretch goal.

A Appendix: Solutions to Concept Checks

Try every question on your own first.

Concept Check 1 (Why Branches Are Catastrophic)

1. 10 cycles represents the depth of the pipeline from fetch to branch-resolution-in-EX, multiplied by issue width. Concretely, with a 5-stage pipeline (fetch, decode, rename, dispatch, execute) and 2-wide fetch, mispredicting wastes $4 \text{ stages} \times 2 \text{ instructions} = 8 \text{ slots}$; round up to 10 for the cost of flushing and restarting. Deeper pipelines (Intel's are 14+ stages) have proportionally higher penalties.
2. Scalar: per-branch penalty in slots = $10/2 = 5$ (since scalar wastes 1 slot per cycle, not 2). With 17% branches at 20% mispredict: $0.17 \times 0.20 \times 5 = 0.17$ wasted slots/inst. $\text{IPC} = 1/1.17 = 0.85$. 2-wide: per-branch penalty in slots = 10. Wasted slots/inst = 0.34. $\text{IPC} = 2/1.34 = 1.49$. Same predictor accuracy, but the 2-wide loses more in absolute IPC (from 2.0 to 1.49 vs scalar 1.0 to 0.85). *Relative* loss is similar, but absolute throughput loss is proportional to width.
3. $0.17 \times 0.01 \times 10 = 0.017$ wasted cycles/inst. $\text{IPC} = 2.0/1.017 = 1.97$. Almost the perfect predictor IPC. This is why 99%-accurate predictors are worth so much.

Concept Check 2 (Static and 1-Bit Dynamic)

1. BTFN predicts backward branches as taken because loops branch backward and loops are taken every iteration except the last. An example of a non-taken backward branch: a continuation point reached by goto that's only triggered in unusual paths (e.g., error recovery from an inner loop).
2. 2 mispredictions per loop execution: one at the loop exit (predictor said T, actual NT, predictor flips to NT), and one on the next entry (predictor still says NT from the last exit, actual T, predictor flips back to T). A perfect predictor would have 1 mispredict per execution (just the loop exit), so the 1-bit predictor doubles the mispredict count compared to perfect.
3. $1024 \text{ entries} \times 1 \text{ bit} = 1024 \text{ bits} = 128 \text{ bytes}$. Trivially small.

Concept Check 3 (2-Bit Counter, BHT, BTB, Speculation)

1. Tracing T T T T NT T T T T from SNT:
 - T: SNT→WNT. Predicted NT, actual T. **Mispredict.**
 - T: WNT→WT. Predicted NT, actual T. **Mispredict.**
 - T: WT→ST. Predicted T, actual T. **Correct.**

- T: ST→ST. Correct.
- NT: ST→WT. Predicted T, actual NT. **Mispredict.**
- T: WT→ST. Predicted T, actual T. Correct.
- T: ST→ST. Correct.
- T: ST→ST. Correct.
- T: ST→ST. Correct.

Total: 3 mispredictions out of 9. The 2-bit predictor handled the single NT outlier without losing direction.

2. PC = 0x1000 = bits[9:2] = 0x00 = index 0. PC = 0x5000 = bits[9:2] = 0x00 = index 0. They alias! Both branches map to BHT[0] and interfere with each other.
3. BHT only stores 2 bits (current state), so a hash collision can return useful information for both branches if their behavior is similar (e.g., both usually taken). BTB stores a full 32-bit target address; if two branches alias and have different targets, the BTB hit returns one of the targets, which is wrong for the other branch — a much more expensive mistake. So BTB usually has tags to verify hits.

Concept Check 4 (The Reservation Station)

1. I2 issues 4 cycles after dispatch (or 3, depending on counting). I1 takes 4 cycles to execute; at cycle T+4, I1's result is in PRF[P32] with ready bit set. I2, sitting in its RS since T+1, sees the ready bit go high in cycle T+4 and issues in cycle T+5 (or T+4 if the wakeup is fast enough to issue same-cycle, which depends on whether wakeup-select is combinational or registered).
2. Storing values would (a) require massive comparators on every CDB broadcast (every RS comparing tags against the CDB), and (b) duplicate the same value across multiple RSs that share a source. Storing only indices keeps the RS small (just $\log_2(\text{PRF size})$ bits per source) and lets the PRF be the single authority on values.
3. The 8 reads of the PRF ready bits go to dedicated read ports on the PRF. In Week 3's design, we gave the PRF 4 read ports for operand reads. For wakeup, we'd need separate (smaller, single-bit) read ports for ready bits. In practice, the wakeup logic doesn't add extra value-read ports; it just exposes the ready-bit array as a combinational signal indexed by all RS source addresses. Real hardware uses a dedicated "ready vector" that the wakeup logic snoops directly.

Concept Check 5 (RS Verilog)

1. The `rs_busy[g]` term ensures only occupied RSs raise wakeup. Without it: a freshly-reset RS has `rs_pj` = 0 and `rs_pk` = 0. The PRF entry for P0 is initially

ready (from Week 3 reset). So both source ready bits are 1, and the wakeup logic would fire on `RS[g]` even though it holds no instruction. The select logic would then issue a garbage instruction, corrupting state.

2. A 4:1 mux of N-bit signals is approximately $2N + \log(4) \cdot N = 3N$ gates per output. For our wider outputs (32 bits for `op + addr + addr + addr = ~ 22` bits total), the total mux is a few hundred gates per output — cheap.
3. RS with the lowest index issues. The other two remain wakeup-ready next cycle (their busy bits stay 1, their sources are still ready). Each cycle the priority encoder picks the next-lowest. So the three RSs issue in order over 3 successive cycles.

Concept Check 6 (RS Testbench)

1. Wakeup logic is fully combinational. `snoop_pj_ready[g]` comes from the PRF combinationaly. `rs_wakeup_ready[g]` ANDs busy with ready. `issue_select[g]` is a combinational function of all wakeup_ready. `issue_valid_o` is combinational. So changing `prf_ready_bits` causes `issue_valid` to update in the same simulation timestep — no clock edge needed. (The actual state change of the RS only happens on the next clock edge, which is why the test waits for `@(posedge clk)` between dispatches.)
2. Because we only have one ALU (in this simplified RS module). Real PRAVAH has 2 ALUs, and the select logic picks one per ALU. To allow multiple issues, we'd need a tree of priority encoders — pick the first wakeup-ready for ALU0, mask it out, pick the next for ALU1. This is the Week 6 extension. For Week 4, single-issue keeps testbenches readable.
3. Because we want to test the RS in isolation. Instantiating the PRF would require driving its writes, allocates, resets — a lot of state. The fake `prf_ready_bits` model is a vector of bits we directly control, letting the test focus on RS behavior. In Week 6 we'll integrate everything together.